Практическая работа №11.

Оценка разрастания таблиц и индексов.

Оценка таблиц и индексов.

Для того, чтобы оценить существующие возможности базы данных необходимо создать новую таблицу в имеющейся базе данных и заполнить ее случайными значениями:

```
postgres=# create extension pgstattuple;
CREATE EXTENSION
postgres=# create table testing (id serial, s text);
CREATE TABLE
postgres=#
postgres=# insert into testing (s) select test.id::text from generate_series
(1,1000000) as test(id);
INSERT 0 1000000
```

Рисунок 1 – Создание новой базы данных и заполнение ее множеством строк.

Первой строкой в коде является подключение расширения pgstattuple, позволяющее узнать сколько полезной, служебной и неактуальной информации содержится в базе. Далее создается индекс для текстового столбца.

```
postgres=# create index on testing (s);
CREATE INDEX
```

Рисунок 2 – Создание индекса для текстового поля.

Далее с помощью команды pgstattuple(‘name_table’) можно узнать необходимую информацию о данных:

```
postgres=# select * from pgstattuple ('testing')\gx
-[ RECORD 1 ]------+---------
table_len          | 44277760
tuple_count        | 1000000
tuple_len          | 34888896
tuple_percent      | 78.8
dead_tuple_count   | 0
dead_tuple_len     | 0
dead_tuple_percent | 0
free_space         | 134412
free_percent       | 0.3
```

Рисунок 3 – Вывод основных параметров по нагрузке таблицы

При этом стоит обратить внимание на следующие параметры:

- tuple_percent — процент полезной информации;
- free_percent — процент свободного пространства (неактуальные строки и пустоты).

В текущем примере tuple_percent – процент полезной информации 78,8%, процент свободного пространства (free_percent) составляет 30%.

Аналогично можно проанализировать индексы на разрастание.

```
postgres=# select * from pgstatindex ('testing_s_idx')\gx
-[ RECORD 1 ]------+---------
version            | 4
tree_level         | 2
index_size         | 22487040
root_block_no      | 290
internal_pages     | 11
leaf_pages         | 2733
empty_pages        | 0
deleted_pages      | 0
avg_leaf_density   | 90.06
leaf_fragmentation | 0
```

Рисунок 4 – Вывод основных операторов по нагрузке индексов

Тут важным параметром является avg_leaf_density, в котором отображается заполненность строк индексами. В текущем случае 90% строк индексов заполнена.

Для того, чтобы проиллюстрировать изменения в таблицах можно обновить половину строк. Предположим, это сделано так:

```
postgres=# update testing set s=s || '??' where id %2=0;
UPDATE 500000
```

Рисунок 5 – Внесение изменений в индексы

И провести аналогичные запросы для анализа таблицы и индексного поля:

```
postgres=# select * from pgstattuple ('testing')\gx
-[ RECORD 1 ]------+---------
table_len          | 66420736
tuple_count        | 1000000
tuple_len          | 35888896
tuple_percent      | 54.03
dead_tuple_count   | 500000
dead_tuple_len     | 17444451
dead_tuple_percent | 26.26
free_space         | 201736
free_percent       | 0.3
```

Рисунок 6 – Иллюстрация изменений в таблице

```
postgres=# select * from pgstatindex ('testing_s_idx')\gx
-[ RECORD 1 ]------+---------
version            | 4
tree_level         | 2
index_size         | 44941312
root_block_no      | 290
internal_pages     | 20
leaf_pages         | 5465
empty_pages        | 0
deleted_pages      | 0
avg_leaf_density   | 75.71
leaf_fragmentation | 49.99
```

Рисунок 7 – Иллюстрация изменений в индексах

Наглядно видны изменения в таблице, какой процент полезной информации и насколько заполнены индексы.

Анализ таблиц.

Также для аналитики таблицы можно использовать также оператор ANALYZE. Для того, чтобы получить отчет по таблицам следует добавить к оператору VERBOSE.

```
postgres=# analyze verbose testing;
ИНФОРМАЦИЯ:  анализируется "public.testing"
ИНФОРМАЦИЯ:  "testing": просканировано страниц: 8108 из 8108, они содержат "
живых" строк: 1000000, "мёртвых" строк: 0; строк в выборке: 30000, примерное
 общее число строк: 1000000
ANALYZE
```

Рисунок 8 – Отработка аналитики таблицы

ANALYZE собирает статистическую информацию о содержимом таблиц в базе данных и сохраняет результаты в системном каталоге pg_statistic. Впоследствии планировщик запросов будет использовать эту статистику для выбора наиболее эффективных планов выполнения запросов.

С указанием VERBOSE команда ANALYZE выдаёт сообщения о процессе анализа, отмечая текущую обрабатываемую таблицу. Также она выводит различные статистические сведения о таблицах. ANALYZE запрашивает для целевой таблицы блокировку только на чтение, так что эта команда может выполняться параллельно с другими операциями с таблицей.

В больших таблицах ANALYZE не просматривает все строки, а обрабатывает только небольшую случайную выборку. Это позволяет проанализировать за короткое время даже очень большие таблицы. Однако учтите, что такая статистика будет лишь приблизительной и может немного меняться при каждом выполнении ANALYZE, даже если фактическое содержимое таблицы остаётся неизменным.

Если у анализируемой таблицы есть один или несколько потомков, ANALYZE соберёт статистику дважды: сначала по строкам только родительской таблицы, а затем по строкам родительской и всех дочерних таблиц. Второй набор статистики необходим для планирования запросов, обращающихся ко всему дереву наследования. Демон автоочистки, однако, принимая решение об автоматическом запуске анализа, будет учитывать операции добавления или изменения данных только в самой родительской таблице. Если именно в этой таблице изменение и добавление происходит редко, наследуемая статистика может терять актуальность, если не запускать ANALYZE вручную.

У данной команды нет аналогов в SQL.

Очистка таблиц.

Помимо анализатора есть еще команды, позволяющие собрать «мусор». Для того, чтобы это сделать есть специальная команда VACUUM. Данная команда может принимать следующие параметры:

FULL – полная очистка. Во время работы блокируется таблица, требует места, т.к. во время выполнения команды потому что происходит запись новой копии таблицы в то время, как старая заблокирована до конца очистки.

VERBOSE – выводит подробный отчет по очистке таблицы.

ANALYZE – обновляет аналитику и статистику.

DISABLE_PAGE_SKIPPING - Обычно VACUUM пропускает страницы, учитывая карту видимости. Страницы, на которых, судя по карте, все кортежи заморожены, можно пропускать всегда, а страницы, в которых все кортежи видны всем транзакциям, могут обрабатываться только при агрессивной очистке. Более того, за исключением агрессивной очистки, некоторые страницы можно пропускать, чтобы не ждать, пока другие сеансы закончат их использовать. Этот параметр отключает пропуск страниц и предназначен для использования только когда целостность карты видимости вызывает подозрения, что возможно при аппаратных или программных сбоях, приводящих к разрушению БД. Каждое отношение таблицы имеет карту видимости (Visibility Map, VM) для отслеживания страниц, содержащих только кортежи, которые видны всем активным транзакциям; в ней также отслеживается, какие страницы содержат только замороженные кортежи.

```
postgres=# vacuum (verbose, analyze) testing;
ИНФОРМАЦИЯ:  очистка "postgres.public.testing"
ИНФОРМАЦИЯ:  закончена очистка "postgres.public.testing": сканирований индек
са: 0
страниц удалено: 0, осталось: 8108, просканировано: 1 (0.01% от общего числа
)
версий строк: удалено: 0, осталось: 1000000, <мёртвых>, но ещё не подлежащих
 удалению: 0
XID отсечки удаления: 773, на момент завершения операции он имел возраст: 0
XID
сканирование индекса не требуется: на страницах таблицы (0, 0.00% от общего
числа) удалено мёртвых идентификаторов элементов: 0
средняя скорость чтения: 0.000 МБ/с, средняя скорость записи: 0.000 МБ/с
использование буфера: попаданий: 9, промахов: 0, <грязных> записей: 0
использование WAL: записей: 0, полных образов страниц: 0, байт: 0
нагрузка системы: CPU: пользов.: 0.00 с, система: 0.00 с, прошло: 0.00 с
ИНФОРМАЦИЯ:  очистка "postgres.pg_toast.pg_toast_16441"
ИНФОРМАЦИЯ:  закончена очистка "postgres.pg_toast.pg_toast_16441": сканирова
ний индекса: 0
страниц удалено: 0, осталось: 0, просканировано: 0 (100.00% от общего числа)
версий строк: удалено: 0, осталось: 0, <мёртвых>, но ещё не подлежащих удале
нию: 0
XID отсечки удаления: 773, на момент завершения операции он имел возраст: 0
XID
новое значение relfrozenxid: 773, оно продвинулось вперёд от предыдущего зна
чения на 8 XID
сканирование индекса не требуется: на страницах таблицы (0, 100.00% от общег
о числа) удалено мёртвых идентификаторов элементов: 0
средняя скорость чтения: 5.157 МБ/с, средняя скорость записи: 5.157 МБ/с
использование буфера: попаданий: 19, промахов: 1, <грязных> записей: 1
использование WAL: записей: 1, полных образов страниц: 0, байт: 188
```

Рисунок 10 – Отработка очистки и аналитики таблицы

В примере у оператора VACUUM указано 2 параметра – VERBOSE и ANALYSE для того, чтобы помимо очистки на экран была выведена информация по аналитике таблиц. Текущая информация может помогать администратору регулярно «чистить» таблицы при накоплении больших данных, в которых некоторые строки являются «битыми».

Также стоит помнить, что ранее были созданы пользователи с разными привилегиями. Для того, чтобы сменить пользователя можно воспользоваться начальной авторизацией в psql, а можно с помощью команды SET SESSION AUTHORIZATION ‘name_user’, чтобы убедиться, что все прошло успешно достаточно просмотреть текущую сессию и текущего пользователя.

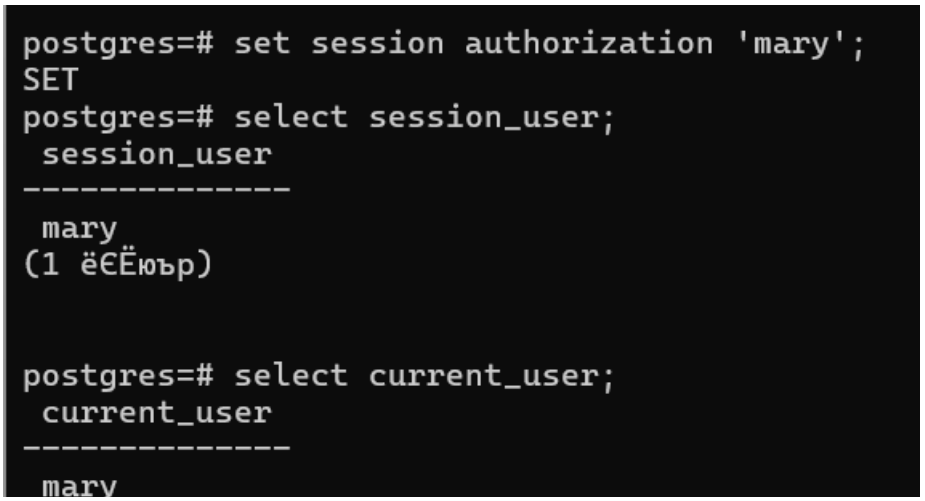


Рисунок 11 – Смена пользователя

При этом пользователь mary создавался, как superuser, что значит, что он имеет все привилегии. Для того, чтобы убедиться в ограниченном функционале возможностей можно переключиться на пользователя student и попробовать сделать с помощью него действия, которые не входят в перечень привилегий.

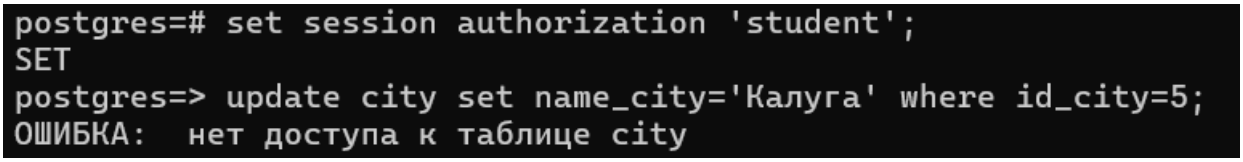


Рисунок 12 – Демонстрация привилегий пользователя.

В выводе команды EXPLAIN для каждого узла в дереве плана отводится одна строка, где показывается базовый тип узла плюс оценка стоимости выполнения данного узла, которую сделал для него планировщик. Если для узла выводятся дополнительные свойства, в вывод могут добавляться дополнительные строки, с отступом от основной информации узла. В самой первой строке (основной строке самого верхнего узла) выводится общая стоимость выполнения для всего плана; именно это значение планировщик старается минимизировать.

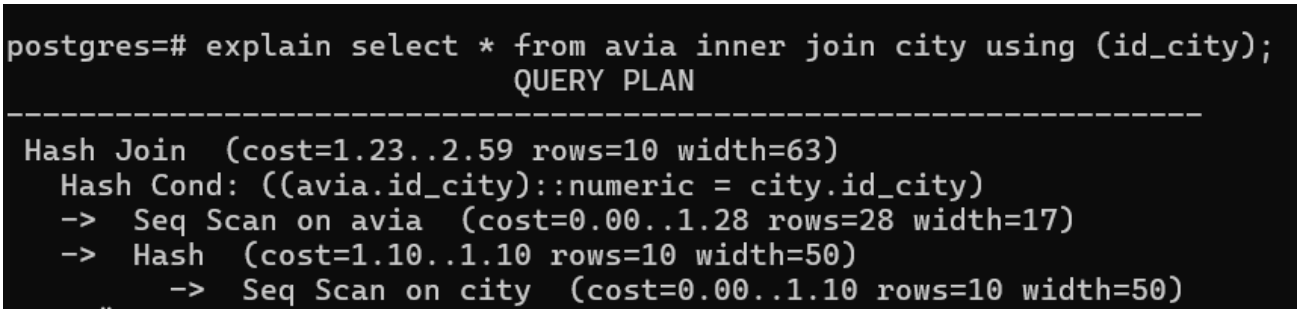


Рисунок 13 – Использование EXPLAIN

Точность оценок планировщика можно проверить, используя команду EXPLAIN с параметром ANALYZE. С этим параметром EXPLAIN на самом деле выполняет запрос, а затем выводит фактическое число строк и время выполнения, накопленное в каждом узле плана, вместе с теми же оценками, что выдаёт обычная команда EXPLAIN.

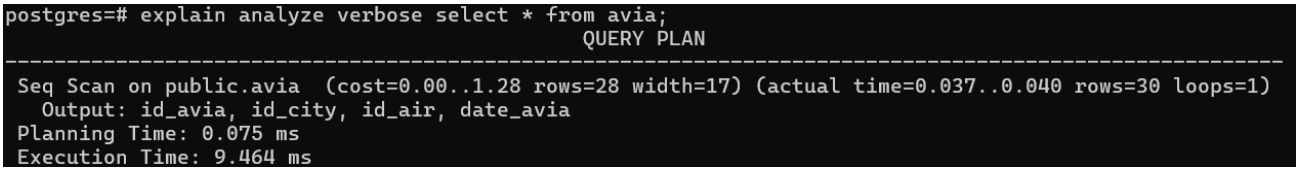


Рисунок 14 – Использование операторов EXPLAIN ANALYZE

Задание.

Необходимо добавить в тестовую таблицу (или уже существующую), более тысячи строк и назначить индекс одному из столбцов.

Далее необходимо исследовать нагрузку таблиц и индексов. Изменить часть индексов (до половины) и сравнить результаты.

Провести аналитику таблицы. Провести очистку «мусора» из таблицы с отчетом.

Сменить пользователя и проверить возможность работы с таблицами пользователя с ограниченным функционалом.

Проанализируйте с помощью представлений из лекций нагрузку на таблицы.

С помощью оператора EXPLAIN проанализируйте разные типы запросов: простые, с JOIN, с вложенной конструкцией. Сделайте выводы. Используйте также ANALYZE совместно с VERBOSE и просмотрите изменения.